There are several quantitation implementations using Apple's Metal Api. All of them works except kernel_mul_mat_q3_k_f32(). Can you find anything wrong with this function?

```
kernel void kernel_mul_mat_q2_k_f32(
        device const  void * src0,
        device const float * src1,
        device       float * dst,
        constant   int64_t & ne00,
        constant   int64_t & ne01,
        constant  uint64_t & nb00,
        constant  uint64_t & nb01,
        constant  uint64_t & nb02,
        constant   int64_t & ne10,
        constant   int64_t & ne11,
        constant  uint64_t & nb10,
        constant  uint64_t & nb11,
        constant  uint64_t & nb12,
        constant   int64_t & ne0,
        constant   int64_t & ne1,
        threadgroup float  * sum [[threadgroup(0)]],
        uint2 tgpig[[threadgroup_position_in_grid]],
        uint2  tpig[[thread_position_in_grid]],              // we
don't use this for now
        uint2 tpitg[[thread_position_in_threadgroup]],
        uint2  tptg[[threads_per_threadgroup]]) {

    const int nb = ne00/QK_K;

    const int64_t r0 = tgpig.x;
    const int64_t r1 = tgpig.y;

    device const block_q2_k * x = (device const block_q2_k
*) src0 + r0*nb;
    device const float     * yy = (device const float      *) src1 +
r1*ne10;
```

```
    const int nth = tptg.x*tptg.y;
    const int ith = tptg.y*tpitg.x + tpitg.y;


    const int tid = tpitg.y;    // 0...16
    const int il  = tid/4;      // 0...3
    const int ir  = tid%4;      // 0...3
    const int ip  = il/2;       // 0 or 1
    const int shift1 = 4*(il%2);// 0 or 4
    const int shift2 = shift1+2;// 2 or 6
    const int n   = 8;
    const int is  = 4*il + (n*ir)/16;

    sum[ith] = 0.0f;

    float sumf = 0;
    for (int i = tpitg.x; i < nb; i += tptg.x) {

        device const uint8_t * q = x[i].qs + 32*ip + n*ir;
        device const uint8_t * scales = x[i].scales + is;

        uint8_t d1 = scales[0] & 0xF;
        uint8_t m1 = scales[0] >>  4;
        uint8_t d2 = scales[2] & 0xF;
        uint8_t m2 = scales[2] >>  4;

        device const float   * y = yy + i*QK_K + 64*il + n*ir;

        const float dall = (float)x[i].d;
        const float dmin = (float)x[i].dmin;

        float4 s = {0.f, 0.f, 0.f, 0.f};
        for (int l = 0; l < n; ++l) {
            s[0] += y[l+ 0] * ((q[l] >> shift1) & 3); s[1] += y[l+ 0];
            s[2] += y[l+32] * ((q[l] >> shift2) & 3); s[3] += y[l+32];
        }
        sumf += dall * (s[0] * d1 + s[2] * d2) - dmin * (s[1] * m1
+ s[3] * m2);
```

```
    }
    sum[ith] = sumf;

    //
    // Accumulate the sum from all threads in the
threadgroup
    // This version is slightly faster than the commented out
one below,
    // which I copy-pasted from ggerganov's q4_0 dot
product for metal.
    //
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%4 == 0) {
        for (int i = 1; i < 4; ++i) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%16 == 0) {
        for (int i = 4; i < 16; i += 4) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith == 0) {
        for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
        dst[r1*ne0 + r0] = sum[0];
    }
}

kernel void kernel_mul_mat_q3_k_f32(
        device const  void * src0,
        device const float * src1,
        device       float * dst,
        constant   int64_t & ne00,
        constant   int64_t & ne01,
        constant  uint64_t & nb00,
        constant  uint64_t & nb01,
        constant  uint64_t & nb02,
        constant   int64_t & ne10,
        constant   int64_t & ne11,
        constant  uint64_t & nb10,
        constant  uint64_t & nb11,
```

```
        constant  uint64_t & nb12,
        constant   int64_t & ne0,
        constant   int64_t & ne1,
        threadgroup float  * sum [[threadgroup(0)]],
        uint2 tgpig[[threadgroup_position_in_grid]],
        uint2  tpig[[thread_position_in_grid]],              // we
don't use this for now
        uint2 tpitg[[thread_position_in_threadgroup]],
        uint2  tptg[[threads_per_threadgroup]]) {

    const uint8_t m1 = 1;
    const uint8_t m3 = 3;
    const int8_t  m4 = 4;

    const uint32_t kmask1 = 0x03030303;
    const uint32_t kmask2 = 0x0f0f0f0f;

    const int nb = ne00/QK_K;

    const int64_t r0 = tgpig.x;
    const int64_t r1 = tgpig.y;

    device const block_q3_k * x = (device const block_q3_k
*) src0 + r0*nb;
    device const float     * yy = (device const float      *) src1 +
r1*ne10;

    const int nth = tptg.x*tptg.y;
    const int ith = tptg.y*tpitg.x + tpitg.y;

    uint32_t utmp[2];

    const int iqs = 16*tpitg.y;
    const int n = iqs/128;                // 0 or 1
    const int r = iqs - 128*n;            // 0...120 in steps of 16
    const int l = 4*(r/16);               // 0...28 in steps of 4
    const int is = l/16;
    const uint8_t m = 1 << (4*n);

    const int shift1 = 4*n;
```

```
    const int shift2 = shift1 + 2;

    float sumf = 0;
    for (int i = tpitg.x; i < nb; i += tptg.x) {

        device const float   * y = yy + i * QK_K + 128*n + l;
        device const uint8_t * q = x[i].qs + 32*n + l;
        device const uint8_t * hm = x[i].hmask + l;

        device const uint32_t * aux = (device const uint32_t
*)x[i].scales;
        utmp[0] = ((aux[0] >> shift1) & kmask2) | (((aux[2] >>
shift1) & kmask1) << 4);
        utmp[1] = ((aux[1] >> shift1) & kmask2) | (((aux[2] >>
shift2) & kmask1) << 4);

        const char4 sc1 = as_type<char4>(utmp[0]);
        const char4 sc2 = as_type<char4>(utmp[1]);

        const float dall = x[i].d;

        float sum = 0;
        for (int k = 0; k < 4; ++k) {
            sum += y[k+ 0] * (sc1[is+0] - 32) * (((q[k] >> 0) & 3) -
(hm[k] & (m << 0) ? 0 : 4))
                + y[k+32] * (sc1[is+2] - 32) * (((q[k] >> 2) & 3) -
(hm[k] & (m << 1) ? 0 : 4))
                + y[k+64] * (sc2[is+0] - 32) * (((q[k] >> 4) & 3) -
(hm[k] & (m << 2) ? 0 : 4))
                + y[k+96] * (sc2[is+2] - 32) * (((q[k] >> 6) & 3) -
(hm[k] & (m << 3) ? 0 : 4));
        }

        sumf += sum * dall;
    }

    sum[ith] = sumf;

    //
    // Accumulate the sum from all threads in the
```

```
threadgroup
    //
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%4 == 0) {
        for (int i = 1; i < 4; ++i) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%16 == 0) {
        for (int i = 4; i < 16; i += 4) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith == 0) {
        for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
        dst[r1*ne0 + r0] = sum[0];
    }

}

static inline uchar4 get_scale_min_k4(int j, device const
uint8_t * q) {
    uchar4 r;
    if (j < 4) {
        r[0] = q[j+0] & 63;
        r[2] = q[j+1] & 63;
        r[1] = q[j+4] & 63;
        r[3] = q[j+5] & 63;
    } else {
        r[0] = (q[j+4] & 0xF) | ((q[j-4] >> 6) << 4);
        r[2] = (q[j+5] & 0xF) | ((q[j-3] >> 6) << 4);
        r[1] = (q[j+4] >>  4) | ((q[j-0] >> 6) << 4);
        r[3] = (q[j+5] >>  4) | ((q[j+1] >> 6) << 4);
    }
    return r;
}

kernel void kernel_mul_mat_q4_k_f32(
        device const  void * src0,
        device const float * src1,
        device       float * dst,
        constant   int64_t & ne00,
```

```
        constant   int64_t & ne01,
        constant  uint64_t & nb00,
        constant  uint64_t & nb01,
        constant  uint64_t & nb02,
        constant   int64_t & ne10,
        constant   int64_t & ne11,
        constant  uint64_t & nb10,
        constant  uint64_t & nb11,
        constant  uint64_t & nb12,
        constant   int64_t & ne0,
        constant   int64_t & ne1,
        threadgroup float  * sum [[threadgroup(0)]],
        uint2 tgpig[[threadgroup_position_in_grid]],
        uint2  tpig[[thread_position_in_grid]],              // we
don't use this for now
        uint2 tpitg[[thread_position_in_threadgroup]],
        uint2  tptg[[threads_per_threadgroup]]) {

    const int nb = ne00/QK_K;

    const int64_t r0 = tgpig.x;
    const int64_t r1 = tgpig.y;

    device const block_q4_k * x = (device const block_q4_k
*) src0 + r0*nb;
    device const float     * yy = (device const float      *) src1 +
r1*ne10;

    const int nth = tptg.x*tptg.y;
    const int ith = tptg.y*tpitg.x + tpitg.y;

    const int tid = tpitg.y;   // 0...16
    const int il  = tid/4;     // 0...3
    const int ir  = tid%4;     // 0...3
    const int n   = 8;
    const int is  = 2*il;

    sum[ith] = 0.0f;

    float sumf = 0;
```

```
    for (int i = tpitg.x; i < nb; i += tptg.x) {

        device const uint8_t * q = (x + i)->qs + 32*il + n*ir;
        device const float   * y = yy + i*QK_K + 64*il + n*ir;
        device const uint8_t * scales = (x + i)->scales;

        const float dall = (float)((x + i)->d);
        const float dmin = (float)((x + i)->dmin);

        const uchar4 sc = get_scale_min_k4(is, scales);

        float4 s = {0.f, 0.f, 0.f, 0.f};
        for (int l = 0; l < n; ++l) {
            s[0] += y[l+ 0] * (q[l] & 0xF); s[1] += y[l+ 0];
            s[2] += y[l+32] * (q[l] >>  4); s[3] += y[l+32];
        }
        sumf += dall * (s[0] * sc[0] + s[2] * sc[2]) - dmin * (s[1] *
sc[1] + s[3] * sc[3]);

    }
    sum[ith] = sumf;

    //
    // Accumulate the sum from all threads in the
threadgroup
    // This version is slightly faster than the commented out
one below,
    // which I copy-pasted from ggerganov's q4_0 dot
product for metal.
    //
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%4 == 0) {
        for (int i = 1; i < 4; ++i) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%16 == 0) {
        for (int i = 4; i < 16; i += 4) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith == 0) {
```

```
        for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
        dst[r1*ne0 + r0] = sum[0];
    }

}

kernel void kernel_mul_mat_q5_k_f32(
        device const  void * src0,
        device const float * src1,
        device       float * dst,
        constant   int64_t & ne00,
        constant   int64_t & ne01,
        constant  uint64_t & nb00,
        constant  uint64_t & nb01,
        constant  uint64_t & nb02,
        constant   int64_t & ne10,
        constant   int64_t & ne11,
        constant  uint64_t & nb10,
        constant  uint64_t & nb11,
        constant  uint64_t & nb12,
        constant   int64_t & ne0,
        constant   int64_t & ne1,
        threadgroup float  * sum [[threadgroup(0)]],
        uint2 tgpig[[threadgroup_position_in_grid]],
        uint2  tpig[[thread_position_in_grid]],              // we
don't use this for now
        uint2 tpitg[[thread_position_in_threadgroup]],
        uint2  tptg[[threads_per_threadgroup]]) {

    const int nb = ne00/QK_K;

    const int64_t r0 = tgpig.x;
    const int64_t r1 = tgpig.y;

    device const block_q5_k * x = (device const block_q5_k
*) src0 + r0*nb;
    device const float     * yy = (device const float      *) src1 +
r1*ne10;

    const int nth = tptg.x*tptg.y;
```

```
    const int ith = tptg.y*tpitg.x + tpitg.y;

    const int tid = tpitg.y;   // 0...16
    const int il  = tid/4;     // 0...3
    const int ir  = tid%4;     // 0...3
    const int n   = 8;
    const int is  = 2*il;

    const uint8_t hm1 = 1u << is;
    const uint8_t hm2 = hm1 << 1;

    float sumf = 0;
    for (int i = tpitg.x; i < nb; i += tptg.x) {

        device const uint8_t * ql = (x + i)->qs + 32*il + n*ir;
        device const uint8_t * qh = (x + i)->qh + n*ir;
        device const float   * y  = yy + i*QK_K + 64*il + n*ir;
        device const uint8_t * scales = (x + i)->scales;

        const float dall = (float)((x + i)->d);
        const float dmin = (float)((x + i)->dmin);

        const uchar4 sc = get_scale_min_k4(is, scales);

        float4 s = {0.f, 0.f, 0.f, 0.f};
        for (int l = 0; l < n; ++l) {
            s[0] += y[l+ 0] * ((ql[l] & 0xF) + (qh[l] & hm1 ? 16 : 0));
s[1] += y[l+ 0];
            s[2] += y[l+32] * ((ql[l] >>  4) + (qh[l] & hm2 ? 16 : 0));
s[3] += y[l+32];
        }
        sumf += dall * (s[0] * sc[0] + s[2] * sc[2]) - dmin * (s[1] *
sc[1] + s[3] * sc[3]);

    }
    sum[ith] = sumf;

    //
    // Accumulate the sum from all threads in the
threadgroup
```

```
    // This version is slightly faster than the commented out one below,
    // which I copy-pasted from ggerganov's q4_0 dot product for metal.
    //
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%4 == 0) {
        for (int i = 1; i < 4; ++i) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith%16 == 0) {
        for (int i = 4; i < 16; i += 4) sum[ith] += sum[ith + i];
    }
    threadgroup_barrier(mem_flags::mem_threadgroup);
    if (ith == 0) {
        for (int i = 16; i < nth; i += 16) sum[0] += sum[i];
        dst[r1*ne0 + r0] = sum[0];
    }
}
```

go over the above code in steps that make sense, don't say as a first pass if you found some errors, just look at them and express some written thoughts that may help you in the second step.

First step first, then you ask me to move on to step two. Be very detailed, and VERY careful

I see that you have posted three kernel functions, namely `kernel_mul_mat_q2_k_f32`, `kernel_mul_mat_q3_k_f32`, and `kernel_mul_mat_q4_k_f32`. You mentioned that there is an issue with `kernel_mul_mat_q3_k_f32`. Let me analyze this particular function.

The code in `kernel_mul_mat_q3_k_f32` is quite complex and dense. It seems to be performing a highly parallelized computation on matrices, potentially with some form of quantization. It is difficult to pinpoint the exact issue without more context or understanding of the expected behavior. However, here are a few pointers that might help in debugging:

1. **Type Casting and Bit Manipulation**: There are several bit manipulations and type casts involved in this kernel. Please double-check if they are as per your intended

behavior. For example, the line `const char4 sc1 = as_type<char4>(utmp[0]);` casts an `uint32_t` to a `char4` type. Make sure the bit manipulations are producing the desired results and not causing unintended side-effects.

2. **Memory Accesses**: Make sure the accesses to the arrays `x`, `y`, `q`, `hm`, etc., are within bounds. Out-of-bounds memory accesses can cause undefined behavior or crashes. In particular, check the calculation of the indices.
3. **Thread Synchronization**: The kernel uses barriers (threadgroup_barriers). Make sure that all threads reach the barrier; otherwise, you can run into deadlocks or race conditions.
4. **Accuracy of Computations**: The kernel seems to be doing fixed-point arithmetic. Be mindful of the precision and rounding errors that might accumulate.
5. **Logic for Condition Checks**: For example, in the lines:

cpp Copy code

```
sum += y[k+ 0] * (sc1[is+0] - 32) * (((q[k] >> 0) & 3) - (hm[k] & (m <<
     + y[k+32] * (sc1[is+2] - 32) * (((q[k] >> 2) & 3) - (hm[k] & (m <<
     + y[k+64] * (sc2[is+0] - 32) * (((q[k] >> 4) & 3) - (hm[k] & (m <<
     + y[k+96] * (sc2[is+2] - 32) * (((q[k] >> 6) & 3) - (hm[k] & (m <<
```

   Make sure that the logic and conditions are correct. Check the parenthesis and order of operations.
6. **Inputs**: Ensure that the input data passed to this kernel function is valid. Sometimes the issue might be with the data that is passed to the function rather than the function itself.
7. **Hardware Limitations**: Make sure that you're not exceeding the hardware limitations of the device it is running on, especially regarding threadgroup sizes and memory usage.

If after checking these points the issue still persists, I would recommend isolating the problematic section by using a process of elimination. You can try using simpler inputs, gradually commenting out code sections to see when it starts working, and print statements (or equivalent) to understand what is happening inside the kernel.

continue to step 2

In the `kernel_mul_mat_q3_k_f32` function, there are a few things that need to be checked for possible issues:

1. Type Casting and Memory Access:
   Make sure that the casting is correct, and also that the memory access patterns are not violating any alignment or range issues.

```cpp
device const block_q3_k * x = (device const block_q3_k *) src0 + r0*nb;
device const float     * yy = (device const float      *) src1 + r1*ne1
```

   Ensure that the structs `block_q3_k` and others are correctly defined elsewhere and match the data you are passing in. Also, check that `src0` and `src1` are pointing to the correct memory locations with the expected data types.
2. Data Preparation:

```cpp
uint32_t utmp[2];
device const uint32_t * aux = (device const uint32_t *)x[i].scales;
utmp[0] = ((aux[0] >> shift1) & kmask2) | (((aux[2] >> shift1) & kmask1
utmp[1] = ((aux[1] >> shift1) & kmask2) | (((aux[2] >> shift2) & kmask1
const char4 sc1 = as_type<char4>(utmp[0]);
const char4 sc2 = as_type<char4>(utmp[1]);
```

   Be careful with bit manipulation and ensure that shifts and masks are correct and that no overflow or unintended data alteration occurs. Also, check that the `as_type` function is used correctly.
3. Arithmetic Operations:

```perl
sum += y[k+ 0] * (sc1[is+0] - 32) * (((q[k] >> 0) & 3) - (hm[k] & (m <<
     + y[k+32] * (sc1[is+2] - 32) * (((q[k] >> 2) & 3) - (hm[k] & (m <<
     + y[k+64] * (sc2[is+0] - 32) * (((q[k] >> 4) & 3) - (hm[k] & (m <<
     + y[k+96] * (sc2[is+2] - 32) * (((q[k] >> 6) & 3) - (hm[k] & (m <<
```

This part contains some complicated arithmetic, ensure that the operations are logically correct and that no unintended casting or rounding occurs. Also, ensure that array indices are within bounds.

4. Thread Synchronization and Memory Barriers:

```scss
threadgroup_barrier(mem_flags::mem_threadgroup);
```

Ensure that all the threads are synchronized properly and memory barriers are used correctly to avoid race conditions.

5. Error Handling:
   - Make sure to check the actual values that are being operated on, perhaps print or log them to see if they are what you expect.
   - Ensure that the kernel is receiving the correct input values and that there isn't an issue elsewhere in your code that might be causing the problem.
   - Check for any compiler warnings or errors that may give a clue as to what's going wrong.
6. Compare with Working Kernels:
   Since you mentioned that other implementations are working except for `kernel_mul_mat_q3_k_f32`, it might be a good idea to compare this function to the other similar functions (e.g., `kernel_mul_mat_q2_k_f32` or `kernel_mul_mat_q4_k_f32`) that are working as expected. Look for any differences in how memory is accessed or arithmetic is performed and see if anything stands out.
7. Unit Testing:
   If possible, try to write unit tests for the smaller parts of the code. This could help isolate which part of the code is not behaving as expected.
8. Documentation and References:
   If this code is based on a particular algorithm or paper, double-check with the original documentation or reference to make sure the implementation is accurate.

Remember that debugging can sometimes be a process of trial and error. Keep making